

CPF | Core Platform Framework

Business Platform Documentation

CPF 프레임워크 개발자 가이드

업무 개발자가 CPF 기능을 선택하고 구현·실패·복구·검증 경로를 빠르게 찾습니다.

문서 기준	Documentation Harness v2.1.0
기준 Source	master 054d894b47f4be8323439dc6f9e58b7d8b60fe54
기준일	2026-08-26

이 문서는 최신 Source 의 공개 계약과 운영 경계를 기준으로 작성하며, 실행하지 않은 검증을 성공으로 기록하지 않습니다.

목차

역할과 질문을 기준으로 필요한 절을 빠르게 찾습니다.

1. 개발 시작과 Domain 생성

- 1.1 환경 확인과 개발 Shell
- 1.2 Domain 생성 · Setup · Sync
- 1.3 Generated/User-owned 경계
- 1.4 기본 Build · Test

2. CRUD 와 Persistence

- 2.1 CRUD 방식 선택
- 2.2 Controller · Service · Repository 기본 흐름
- 2.3 JDBC · MyBatis · JPA 선택
- 2.4 Paging · Bulk · Lock · Data Quality
- 2.5 Transaction 연결과 DB3 주의

3. Transaction · Idempotency · Reconcile

- 3.1 Local Transaction 과 REQUIRES_NEW
- 3.2 Remote Side Effect
- 3.3 Idempotency 와 Retry
- 3.4 UNKNOWN 과 Reconcile
- 3.5 Saga · TCC · XA 선택

4. Domain 과 외부 시스템 호출

- 4.1 Same JVM/Remote Domain Invocation
- 4.2 System6 · operationId · Timeout
- 4.3 HTTP · Fixed-Length · GraphQL · Realtime · Webhook
- 4.4 External Failure · Retry · UNKNOWN
- 4.5 AI Provider 연계

5. Messaging · Notification

- 5.1 Provider 선택
- 5.2 Producer/Consumer 와 Event
- 5.3 Outbox · Inbox · Duplicate · DLQ

- 5.4 Notification Channel · Receipt

- 5.5 Retry · Replay · Idempotency

6. Cache · File · Framework Common

- 6.1 Cache · TTL · Invalidation · Multi-instance
- 6.2 File/Object Storage · Archive · Tabular
- 6.3 Validation · Code/Parameter/Message 공통 기능
- 6.4 Provider Escape Hatch 와 장애 처리

7. Security · Approval · Audit

- 7.1 Authentication · Authorization
- 7.2 Masking · Secret · Digital Signature
- 7.3 Approval · Reason · Audit
- 7.4 실패/권한 Test

8. Context · Error · Trace

- 8.1 Context/Header 규칙
- 8.2 Result · Business/Technical/Validation Error
- 8.3 Log · Trace · Timeline
- 8.4 Runtime instance/operation 추적

9. Starter · Config · Generator

- 9.1 Starter/Capability 선택
- 9.2 Profile · Provider · Configuration
- 9.3 Generator/Sync 와 삭제 승인
- 9.4 OpenAPI/Frontend/Platform Operations 연계

10. Test 와 운영 인계

- 10.1 Unit/Integration/Runtime Test
- 10.2 실패 · 경계 · 복구 Test
- 10.3 운영 인계 식별자/Runbook
- 10.4 금지 패턴과 Source/EDU 길찾기

1. 개발 시작과 Domain 생성

1.1 환경 확인과 개발 Shell

개발을 시작할 때는 Java 25, root Gradle, cpf CLI 와 현재 Source identity 를 먼저 확인합니다. 이후 Domain 생성 · Build · Test 를 같은 Source 기준으로 실행해 환경 차이를 조기에 분리합니다.

```
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 status
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 build
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 verify-full
```

1.2 Domain 생성·Setup·Sync

Domain lifecycle 은 cpf domain create/setup/sync/diff/remove 와 verify 흐름으로 관리합니다. Generator 가 Canonical Catalog/계약을 기준으로 생성하고 사용자 소유 Source 를 임의 삭제하지 않습니다.

- 변경 전 diff 와 ownership 을 확인합니다.
- 삭제가 필요한 경우 exact manifest/승인 절차를 거쳐 Generated/User-owned 경계를 보존합니다.

```
python cpf-tools/runtime/cli/cpf.py domain --help
python cpf-tools/runtime/cli/cpf.py domain create --help
python cpf-tools/runtime/cli/cpf.py verify domain --help
```

1.3 Generated/User-owned 경계

Generated Domain 은 Project→Runtime Module→Java Base Package→Business Feature→Technical Role 순서로 구조를 분리합니다. Runtime 이름이나 Domain 이름을 package/feature 에 중복 생성하지 않습니다.

- base/는 Generator-owned bootstrap/common contract 에만 사용합니다.
- Feature 에 필요하지 않은 controller/service/repository role 을 기계적으로 만들지 않습니다.

1.4 기본 Build·Test

기본 검증은 root Gradle configuration→compile→test 순서로 실행하고, Generator/Starter 를 바꿨다면 실제 Generated Domain consumer 까지 같은 Source 에서 빌드합니다.

- 현재 baseline 은 Java 25, Gradle 9.1.0, Spring Boot 4.1.0 입니다.
- 공개 Surface 변경은 consumer compile/runtime 과 generated output 까지 회귀 검증합니다.

하려는 일에 맞는 CPF 기능과 선택 조건을 고릅니다.

계약과 실제 Consumer 를 확인할 Source 위치를 찾습니다.

Source 길찾기 [cpf-tools/runtime/cli/cpf.py](#)

2. CRUD 와 Persistence

2.1 CRUD 방식 선택

CRUD 는 Controller 가 거래 경계를 받고 Service 가 업무 규칙과 Transaction 을 소유하며 Repository 가 persistence 를 담당하는 흐름을 기본으로 합니다. JDBC·MyBatis·JPA 는 팀 표준과 Query 특성에 따라 선택합니다.

2.2 Controller·Service·Repository 기본 흐름

Persistence 는 JDBC/MyBatis/JPA 중 업무와 팀 표준에 맞는 Provider 를 선택하면서 Repository·Paging·Bulk·Lock 의 공통 계약을 유지합니다.

- Domain 은 다른 Owner 의 Repository/DB 를 직접 조회하지 않습니다.
- Bulk/Paging 은 transaction 범위·lock·index·memory pressure 를 함께 검토합니다.

2.3 JDBC·MyBatis·JPA 선택

JDBC 는 직접 SQL 제어가 필요한 경우, MyBatis 는 명시적 Mapper/SQL 중심 개발, JPA 는 Entity lifecycle 과 ORM 이 적합한 경우 선택합니다. 어떤 Provider 를 쓰더라도 Owner DB 경계와 CPF Transaction 계약은 유지합니다.

2.4 Paging·Bulk·Lock·Data Quality

Paging 과 Bulk 는 page size·transaction 범위·lock·index·memory 사용량을 함께 설계하고, 동시 수정 가능성이 있으면 optimistic/pessimistic lock 과 재시도 조건을 명확히 합니다.

2.5 Transaction 연결과 DB3 주의

Transaction 과 Persistence 변경은 Oracle·PostgreSQL·MariaDB 의 Schema/Query 영향을 함께 확인합니다. 한 Vendor 의 로컬 성공만으로 DB3 호환성을 확정하지 않습니다.

공식 DB Vendor 는 Oracle·PostgreSQL·MariaDB 만 사용하며 세 Vendor 의 DDL·Migration·Rollback·Runtime 동작을 함께 검증합니다.

- Vendor 차이는 datatype·identity·locking·DDL syntax 처럼 실제 동작 차이가 있는 지점에서만 분기합니다.

하려는 일에 맞는 CPF 기능과 선택 조건을 고릅니다.

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

Source 길찾기 [cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/transaction/CpfTransactionOperations.java](https://github.com/cpf-starters/data/persistence/src/main/java/com/cpf/data/persistence/api/transaction/CpfTransactionOperations.java)

3. Transaction·Idempotency·Reconcile

그림 3-1. Transaction · UNKNOWN · Reconcile



Local Transaction 과 Remote Side Effect 를 분리하고 UNKNOWN 을 Reconcile 로 확정하는 흐름입니다.

3.1 Local Transaction 과 REQUIRES_NEW

Local Transaction 은 동일 DB/Resource 안에서 원자성을 보장하는 범위로 사용합니다. REQUIRED 와 REQUIRES_NEW 선택은 업무 경계와 독립 커밋 필요성으로 판단합니다.

- Remote Side Effect 를 Local DB Transaction 성공과 동일시하지 않습니다.
- 독립 감사/이력 커밋이 본 거래 rollback 의미를 왜곡하지 않도록 경계를 검토합니다.

3.2 Remote Side Effect

분산 Side Effect 는 Local Transaction 으로 감출 수 없습니다. Saga/TCC/XA 는 참여 Resource 와 복구 모델에 따라 선택 하고, 결과 불명확 시 보상보다 먼저 Reconcile 가능성을 판단합니다.

- 동일 요청의 재시도는 idempotency key 와 상태 저장으로 안전하게 만듭니다.
- Compensation 은 이미 확정된 Side Effect 를 되돌리는 별도 업무 동작으로 추적합니다.

3.3 Idempotency 와 Retry

Idempotency 는 동일 의미의 요청이 재전송돼도 Side Effect 가 한 번만 확정되도록 durable 상태로 관리합니다. 메모리 flag 만으로 다중 인스턴스 중복을 막지 않습니다.

- idempotency key 는 업무 Operation · 호출자 범위와 결과 보존 기간을 함께 정의합니다.
- 처리 중 Process Kill 후에도 재시도가 같은 결과를 재사용하거나 안전하게 복구해야 합니다.

3.4 UNKNOWN 과 Reconcile

UNKNOWN 은 결과를 확인할 Evidence 가 부족한 정상적인 운영 상태입니다. Reconcile 은 외부기관·DB·Broker 의 실제 결과를 조회해 상태를 확정하거나 manual review 로 넘깁니다.

- UNKNOWN 에서 무조건 재호출하면 중복 Side Effect 가 생길 수 있습니다.
- Reconcile 결과·actor·reason·시각을 추적 가능하게 남깁니다.

3.5 Saga·TCC·XA 선택

Saga·TCC·XA 는 참여 자원과 보상·확정 방식이 다릅니다. 장기 업무 흐름은 Saga, Try/Confirm/Cancel 은 TCC, XA 참여 자원은 XA 를 선택하고 원격 결과가 불명확하면 먼저 Reconcile 합니다.

- 동일 요청의 재시도는 idempotency key 와 상태 저장으로 안전하게 만듭니다.
- Saga compensation 은 원래 업무와 별도 Operation/Trace 로 남겨 재보상과 중복 실행을 구분합니다.

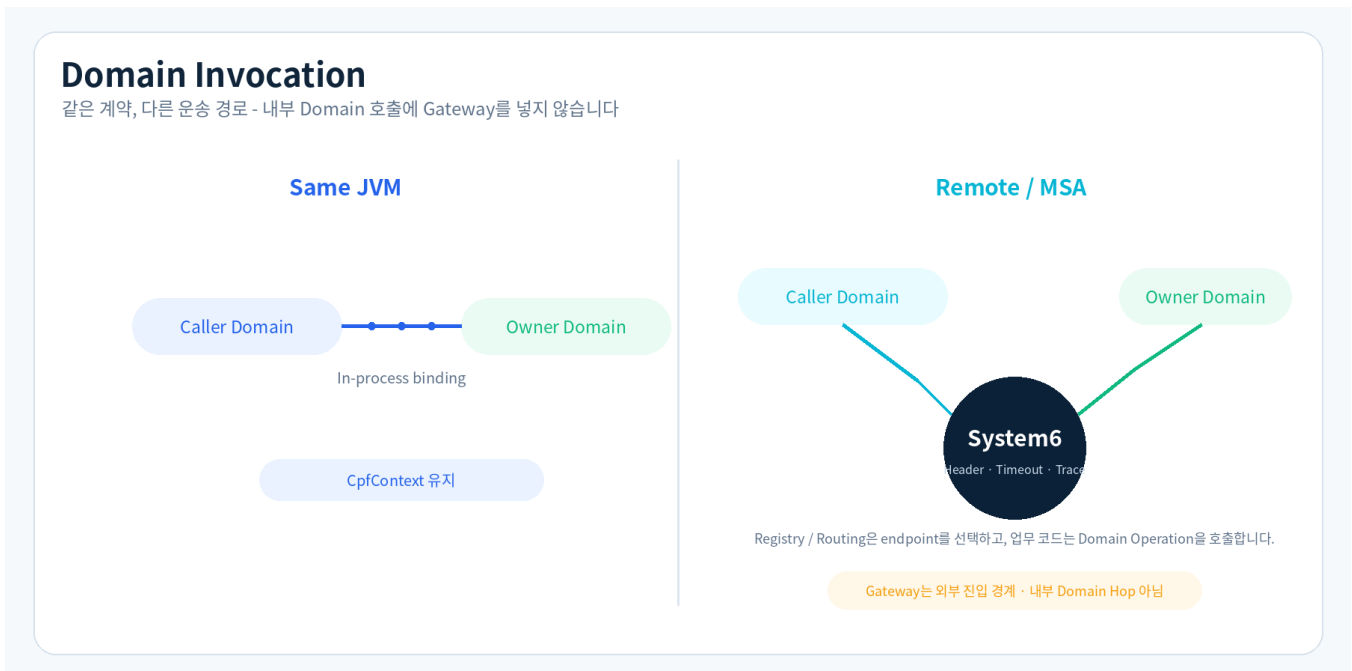
상황에 맞는 선택과 피해야 할 경로를 판단합니다.

기본값이 아닌 실제 선택이 필요한 옵션을 확인합니다.

Source [길찾기 cpf-core/src/main/java/com/cpf/core/api/transaction](#)

4. Domain 과 외부 시스템 호출

그림 4-1. Domain Invocation



Same JVM 과 Remote 가 같은 업무 호출 계약을 유지하며 내부 Domain 호출은 Gateway 를 경유하지 않습니다.

4.1 Same JVM/Remote Domain Invocation

업무 Source 는 배치 방식에 따라 local/remote 분기를 작성하지 않습니다. 공식 Domain Invocation 계층이 Same JVM 과 Remote route 를 선택하면서 동일한 operation·context·error 의미를 유지합니다.

- Same JVM 은 CpfContext 계열 논리 Context 를 사용하고 self-HTTP 를 하지 않습니다.

- Remote 는 System6, timeout/deadline, retry, trace 와 UNKNOWN/reconcile 계약을 경계에서 적용합니다.

4.2 System6·operationId·Timeout

업무 Online Transaction 은 Controller 실행 전에 Canonical System6 를 갖습니다. Transaction/Original 은 유지하고 System/Caller/Target/Operation 은 Hop 기준으로 CPF 가 확정합니다.

- Browser 가 Protected Header 를 보내도 신뢰하지 않고 Trusted Entry 에서 재구성합니다.
- 수신 System/Target/Operation 이 실제 Runtime/Handler 와 다르면 업무 Controller 를 호출하지 않습니다.

4.3 HTTP·Fixed-Length·GraphQL·Realtime·Webhook

External HTTP 호출은 timeout, deadline, credential, retry 조건을 호출 계약에 포함합니다. 응답을 받지 못한 상태에서는 원격 Side Effect 의 성공 여부를 추정하지 않습니다.

- retry 는 idempotent operation 또는 명시적 idempotency 계약에서만 허용합니다.
- credential·secret 을 URL/log/evidence 에 노출하지 않습니다.

4.4 External Failure·Retry·UNKNOWN

외부 호출 실패에서 응답 유실·timeout 은 대상 Side Effect 가 수행됐을 수 있으므로 UNKNOWN 으로 분리합니다. Retry 전에 target probe 나 Reconcile 로 실제 결과를 확인합니다.

- UNKNOWN 에서 무조건 재호출하면 중복 Side Effect 가 생길 수 있습니다.
- Reconcile 결과·actor·reason·시각을 추적 가능하게 남깁니다.

4.5 AI Provider 연계

AI Provider 연계는 모델 호출 자체보다 policy, risk, telemetry, 민감정보 처리와 결과 확정 의미를 함께 다룹니다. timeout·실패·UNKNOWN 을 일반 업무 성공으로 감추지 않습니다.

배포 방식이 달라도 유지해야 할 호출 계약을 비교합니다.

표 4-1. 관점 / Same JVM / Remote / 공통 계약

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

관점	Same JVM	Remote	공통 계약
호출 경로	In-process route	Registry+transport	Domain Invocation
Context	CpfContext	System6 serialize	동일 논리 Context
실패	직접 예외/Result	timeout·network	표준 Error/UNKNOWN
관측	동일 trace	hop trace	transaction/operation 유지

각 Header 의 설정 주체·변경 규칙·수신 검증을 확인합니다.

표 4-2. Header / 설정 주체 / 변경 규칙 / 검증/주의

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

Header	설정 주체	변경 규칙	검증/주의
X-Transaction-Id	CPF	최초 생성 후 불변	거래 전체 correlation
X-Original-System-Code	CPF Trusted Entry	최초 확정 후 불변	호출자가 임의 변경 금지
X-System-Code	CPF	Hop 마다 현재 처리 System	수신 Runtime 과 일치
X-Caller-System-Code	CPF	Hop 마다 직전 Caller	신뢰 System identity 사용
X-Target-System-Code	CPF	Hop 마다 Target	수신 Runtime 과 일치
X-Target-Operation-Id	CPF	Operation 마다 갱신	실제 Handler 와 일치

Source [길찾기 cpf-starters/integration/ai/src/main/java/com/cpf/integration/ai/CpfAiRouter.java](#)

5. Messaging·Notification

5.1 Provider 선택

Messaging Provider 는 Broker/전송 구현을 선택하는 확장점입니다. Producer·Consumer 계약, schema, duplicate, DLQ, replay 의미는 Provider 가 바뀌어도 업무 코드에서 일관되게 유지합니다.

5.2 Producer/Consumer 와 Event

Messaging 은 Broker 종류와 무관하게 schema, producer/consumer context, duplicate, DLQ 와 replay 를 함께 관리합니다. 전송 성공과 업무 처리 성공을 같은 상태로 보지 않습니다.

- Outbox/Inbox 를 사용할 때 DB commit 과 publish/consume 상태를 분리합니다.
- Replay 는 idempotency 와 ordering/partition 영향을 확인한 뒤 수행합니다.

5.3 Outbox·Inbox·Duplicate·DLQ

Outbox/Inbox 는 DB commit 과 message publish/consume 상태를 분리해 중복·재전송을 안전하게 다룹니다. DLQ 는 실패 보관소이지 자동 성공 처리 경로가 아니며 replay 전에 idempotency 와 ordering 을 확인합니다.

- Inbox/Outbox dedup key 와 Event 보존 기간을 replay 가능한 범위와 맞춥니다.

5.4 Notification Channel·Receipt

Notification 은 발송 요청, Provider 결과, Receipt, Reconcile 상태를 분리합니다. 요청 수락만으로 최종 전달 성공을 확정하지 않습니다.

- Provider 별 message/receipt ID 를 내부 execution 과 연결합니다.
- UNKNOWN/지연 상태는 중복 발송 전에 조회·Reconcile 합니다.

5.5 Retry·Replay·Idempotency

Messaging 재시도와 Replay 는 같은 동작이 아닙니다. Retry 는 일시 실패를 다시 시도하고 Replay 는 과거 Event 를 다시 처리하므로 idempotency key, ordering, partition 과 기존 Side Effect 를 먼저 확인합니다.

- Replay 식별자는 원 Event·partition·consumer 처리 결과를 다시 추적할 수 있어야 합니다.

하려는 일에 맞는 CPF 기능과 선택 조건을 고릅니다.

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

Source [길찾기 cpf-starters/base/runtime/src/main/java/com/cpf/reliability/runtime/CpfIdempotencyCoordinator.java](#)

6. Cache·File·Framework Common

6.1 Cache·TTL·Invalidation·Multi-instance

Cache 는 get/put/evict 뿐 아니라 TTL, invalidation, multi-instance refresh, stale/failure, reconnect, stampede, serialization 과 Provider 충돌을 함께 다룹니다. 인스턴스별 메모리 값만으로 전역 최신성을 가정하지 않습니다.

- 다중 인스턴스에서는 invalidation 전과 실패와 stale cache 를 감지·복구할 수 있어야 합니다.
- Provider 장애 시 원장 우선순위와 fallback 동작을 명확히 하고 stampede 를 제한합니다.

6.2 File/Object Storage·Archive·Tabular

File 기능은 Attachment, Object Storage, Archive/Checksum/Tabular 처리를 선택형 Capability 로 제공합니다. 파일 원문과 metadata, scan/검증 상태를 분리합니다.

- checksum·content type·size·inspection 결과를 처리 전 검증합니다.
- Object Storage 장애/partial upload 는 reconcile/cleanup 경로를 가집니다.

6.3 Validation·Code/Parameter/Message 공통 기능

Framework Common 은 Validation, Code/Parameter, Message 같은 반복 기능을 공통 계약으로 제공합니다. 업무 Domain 은 동일한 검증·코드 조회·메시지 규칙을 개별 구현으로 복제하지 않습니다.

- 공통 기능 변경은 실제 업무 Consumer 와 EDU/Sample 에서 호출되는지 확인합니다.

6.4 Provider Escape Hatch 와 장애 처리

공식 Provider 가 모든 고급 기능을 가릴 필요는 없습니다. Native Spring/OSS API 가 더 적합한 경우 escape hatch 를 제공하되 Context, 보안, Trace 와 실패 의미를 우회하지 않습니다.

하려는 일에 맞는 CPF 기능과 선택 조건을 고릅니다.

필요 Capability 에 맞는 Starter/Provider 를 선택합니다.

표 6-1. 필요 기능 / Starter/Provider / 설정 / 선택 기준

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

필요 기능	Starter/Provider	설정	선택 기준
Web API	cpf-starter-web-api	HTTP/Context	Online API

필요 기능	Starter/Provider	설정	선택 기준
Secure API	cpf-starter-secure-api	Security	보호 API
Persistence	data-jdbc/mybatis/jpa	Provider 1 개	DB 접근
Messaging	kafka/rabbitmq/jms/ibm-mq	Broker 선택	비동기 연계
Cache	caffeine/redis/valkey	TTL/invalid.	cache 필요 시

[Source](#) [길찾기](#) [cpf-starters/data/cache](#)

7. Security·Approval·Audit

7.1 Authentication·Authorization

Security 는 Authentication 이후 Authorization/Permission 을 분리해 평가하고 Session/Token lifecycle 을 Runtime 경계에 맞춥니다. 관리 기능과 업무 기능은 서로 다른 권한 Surface 를 가집니다.

- Protected Header 와 사용자 credential 을 동일 identity 로 취급하지 않습니다.
- 401/403 은 원인을 구분하고 실패도 audit/trace 와 연결합니다.

7.2 Masking·Secret·Digital Signature

Crypto/Privacy 기능은 secret·key·certificate lifecycle 과 masking 을 함께 관리합니다. 민감정보 원문은 log, 화면, test/evidence 에 남기지 않는 것이 기본입니다.

- Signature 는 key rotation/rekey 와 verification failure 를 별도 상태로 다룹니다.
- Masking 해제나 민감정보 조회는 permission·reason·audit 가 필요합니다.

7.3 Approval·Reason·Audit

위험 운영 조치는 Permission 만으로 끝내지 않고 reason, 필요 시 approval, 실행 결과와 audit 를 연결합니다. Break-glass 는 일반 권한 우회가 아니라 별도 통제된 비상 절차입니다.

- 승인자와 실행자의 SoD 를 적용할 수 있어야 합니다.
- 실행 실패도 승인/요청/결과 trace 에서 누락하지 않습니다.

7.4 실패/권한 Test

Security Test 는 인증 없음, 권한 부족, 만료/변조 credential, 보호 Header 위조, 승인 필요 조치와 감사 기록까지 포함합니다. 401 과 403 을 같은 실패로 묶개지 않습니다.

- 권한 실패가 Controller 업무 로직 실행 전에 차단되는지 Runtime Test 로 확인합니다.
- Source identity, 명령, ExitCode, Evidence 를 같은 실행 결과에 연결합니다.

작업에 사용할 Public API 와 실패 주의점을 확인합니다.

기본값이 아닌 실제 선택이 필요한 옵션을 확인합니다.

[Source](#) [길찾기](#) [cpf-starters/security/src/main/java/com/cpf/security/api/approval/CpfApprovalVerifier.java](#)

8. Context·Error·Trace

그림 8-1. Operations Trace Timeline



transactionId·operationId·instanceId 를 기준으로 오류와 Runtime 을 같은 Timeline 에서 추적합니다.

8.1 Context/Header 규칙

Context 경계에서는 Canonical System6 의 불변/가변 필드를 구분합니다. Transaction/Original 은 거래 전체에서 유지하고 Caller/Target/Operation 은 현재 Hop 기준으로 재검증합니다.

- 외부 입력의 Protected Header 는 Trusted Entry 에서 제거·재구성하고 내부 Context 와 일치하는지 검증합니다.
- Target System/Operation 과 실제 Handler 불일치는 Controller 전 단계에서 거절하고 Trace 에 남깁니다.

8.2 Result·Business/Technical/Validation Error

CPF 는 정상 결과와 Business/Technical/Validation 실패를 표준 Result/Error 의미로 구분합니다. 외부 Side Effect 결과가 확정되지 않으면 성공이나 실패로 추정하지 않고 UNKNOWN 으로 유지합니다.

- Business 실패는 정상 업무 판단과 구분해 코드/메시지를 전달합니다.
- Technical 실패는 retry 가능성·trace 식별자·운영 조치와 연결합니다.

8.3 Log·Trace·Timeline

Observability 는 transactionId·operationId·instanceId 를 중심으로 Log, Trace, Metric, Timeline 을 연결합니다. 운영자는 코드 위치보다 거래·실행 식별자에서 시작해 원인을 좁힐 수 있어야 합니다.

- 민감정보는 structured log 에서도 마스킹합니다.
- 재시도/복구 segment 가 원 거래와 단절되지 않도록 correlation 을 유지합니다.

8.4 Runtime instance/operation 추적

instanceId 는 현재 Process 를 식별하는 Runtime 축이며 System6 Header 에는 포함하지 않습니다. 기동 시 설정→환경 변수→Hostname 순으로 한 번 확정하고 Process 수명 동안 유지합니다.

- localhost/unknown/Domain 명 같은 fallback 으로 READY 를 허용하지 않습니다.
- 동일 Host 의 동일 System 다중 Process 는 explicit instanceId 가 필요합니다.

각 Header 의 설정 주체·변경 규칙·수신 검증을 확인합니다.

표 8-1. Header / 설정 주체 / 변경 규칙 / 검증/주의

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

Header	설정 주체	변경 규칙	검증/주의
X-Transaction-Id	CPF	최초 생성 후 불변	거래 전체 correlation
X-Original-System-Code	CPF Trusted Entry	최초 확정 후 불변	호출자가 임의 변경 금지
X-System-Code	CPF	Hop 마다 현재 처리 System	수신 Runtime 과 일치
X-Caller-System-Code	CPF	Hop 마다 직전 Caller	신뢰 System identity 사용
X-Target-System-Code	CPF	Hop 마다 Target	수신 Runtime 과 일치
X-Target-Operation-Id	CPF	Operation 마다 갱신	실제 Handler 와 일치

오류 상태에서 Retry·복구·확인 순서를 판단합니다.

Source 길찾기 [cpf-starters/platform-operations/src/main/java/com/cpf/platform/operations/api/runtime/CpfInstanceIdentity.java](#)

9. Starter·Config·Generator

9.1 Starter/Capability 선택

Starter 는 필요한 Capability 를 명시적으로 선택하는 공개 진입점입니다. 선택하지 않은 Capability 가 bean·thread·config·SQL·endpoint Side Effect 를 남기지 않아야 합니다.

- 한 Deployable 은 기본적으로 exactly-one Top-level Profile 을 선택합니다.
- 선택하지 않은 Optional Capability 는 bean/thread/config/sql/endpoint Side Effect 가 없어야 합니다.

9.2 Profile·Provider·Configuration

Profile 은 Deployable 형태를, Provider 는 Capability 구현체를, Configuration 은 동작 옵션을 결정합니다. 서로 충돌하는 Provider/Profile 은 기동 시 fail-fast 하고 secret 은 일반 설정 출력에 노출하지 않습니다.

- Profile 은 Deployable 역할을 하나로 고정하고 Provider 는 Capability 별 구현 선택으로 분리합니다.
- Provider 미선택·충돌은 조용한 fallback 보다 fail-fast 또는 명시적 비활성으로 처리합니다.

9.3 Generator/Sync 와 삭제 승인

Generator 변경은 create/setup/sync 결과와 diff 를 먼저 확인하고, 생성영역을 넘어선 사용자 Source 는 자동 덮어쓰기 · 삭제 대상으로 취급하지 않습니다.

- 변경 전 diff 와 ownership 을 확인합니다.
- remove/sync 가 삭제를 요구하면 대상 경로와 ownership 을 먼저 보여주고 승인된 범위만 반영합니다.

9.4 OpenAPI/Frontend/Platform Operations 연계

OpenAPI 는 Controller 계약의 operationId 와 오류/DTO 를 외부 Consumer Surface 로 고정합니다. Frontend 는 Generated Client 를 실제 화면 Consumer 에 연결하고 수기 중복 API wrapper 를 줄입니다.

- 401·403·404·409·429·500·503 상태를 화면/운영 흐름에서 구분합니다.
 - Generated client 가 stale 하지 않도록 Source→OpenAPI→client→route coverage 를 검증합니다.
- 필요 Capability 에 맞는 Starter/Provider 를 선택합니다.

표 9-1. 필요 기능 / Starter/Provider / 설정 / 선택 기준

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

필요 기능	Starter/Provider	설정	선택 기준
Web API	cpf-starter-web-api	HTTP/Context	Online API
Secure API	cpf-starter-secure-api	Security	보호 API
Persistence	data-jdbc/mybatis/jpa	Provider 1 개	DB 접근
Messaging	kafka/rabbitmq/jms/ibm-mq	Broker 선택	비동기 연계
Cache	caffeine/redis/valkey	TTL/invalid.	cache 필요 시

기본값이 아닌 실제 선택이 필요한 옵션을 확인합니다.

[Source 길찾기 cpf-admin/frontend/src/generated/orval](#)

10. Test 와 운영 연계

10.1 Unit/Integration/Runtime Test

Unit 은 계약 단위, Integration 은 Provider/DB/외부 경계, Runtime 은 실제 Consumer→Side Effect→Trace 까지 검증합니다. 실행하지 않은 단계는 PASS 로 기록하지 않습니다.

- 미실행 단계와 환경 제약은 PASS 가 아니라 미검증으로 분리합니다.
- 검증 결과에는 기준 Source, 실행 명령, ExitCode 와 실패 단계가 함께 남아야 합니다.

10.2 실패·경계·복구 Test

실패 Test 는 timeout, duplicate, partial failure, UNKNOWN, Process Kill 과 재실행까지 포함해 복구 후 상태와 Side Effect 가 일관되는지 확인합니다.

- 실패 주입 뒤 복구 결과까지 실행하지 않았다면 복구 PASS 로 기록하지 않습니다.

- 복구 성공 여부는 동일 식별자의 최종 상태와 Side Effect 를 다시 확인해 판정합니다.

10.3 운영 인계 식별자/Runbook

운영 인계에는 transactionId · operationId · instanceId, 주요 오류 상태, 재시도/재처리 조건, Source 위치와 검증 명령을 연결해 장애 시 코드 위치부터 찾지 않아도 되게 합니다.

10.4 금지 패턴과 Source/EDU 길찾기

금지 패턴은 Owner · Topology · 신뢰 경계를 우회하거나 운영 Evidence 를 잃게 만드는 구현을 빠르게 차단하기 위한 Review Gate 입니다.

- self-HTTP, 타 Owner DB/Internal 직접 참조, secret 원문 로그를 금지합니다.
- 미실행 검증 · stale Evidence · Optional 기능의 숨은 Side Effect 를 완료 근거로 사용하지 않습니다.

정상 · 실패 · 복구까지 통과해야 할 시나리오를 확인합니다.

계약과 실제 Consumer 를 확인할 Source 위치를 찾습니다.

Source 길찾기 [cpf-tools/runtime/cli/cpf.py](https://github.com/CPF-Tools/runtime/cli/cpf.py)